

第四章 消息认证

顾辰

guchen@hfut.edu.cn



为什么需要消息认证

- ❑ 在实际的交易中对称加密或公钥加密还有什么问题没有解决？
- ❑ 以银行转账场景为例，银行A向银行B转账。
 - 如何保证支付信息未被篡改？
 - 如何保证买方/卖方身份的真实性？

对称加密或公钥加密并不能解决认证的问题

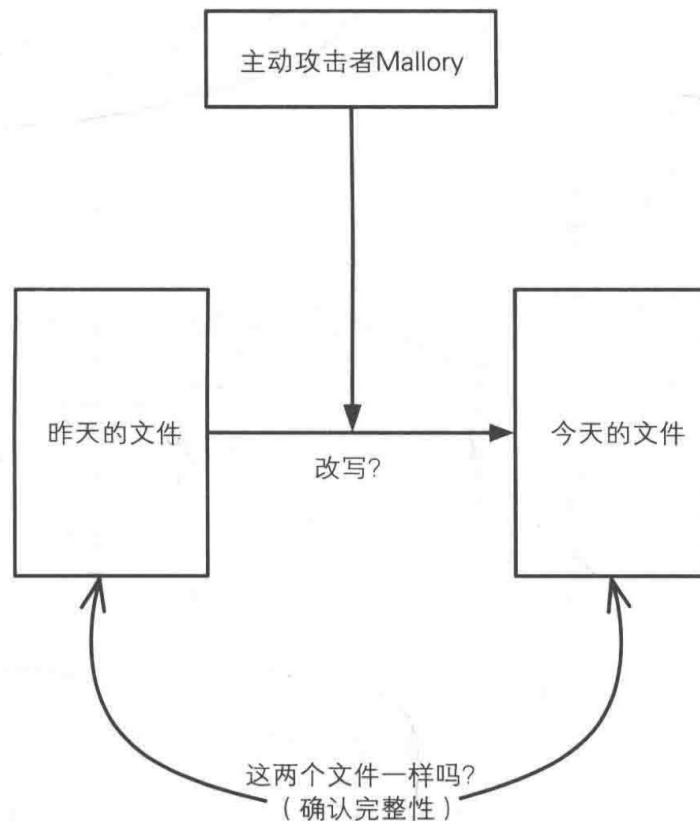
4.1 单向散列函数

□ 如何确保文件未被篡改？

➤ 大小

➤ 修改时间

□ 如何用**最小**的代价确保文件的完整性？

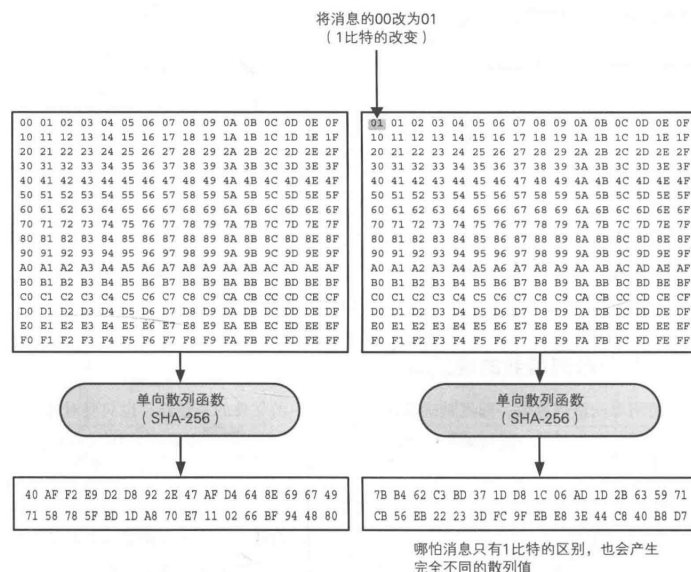
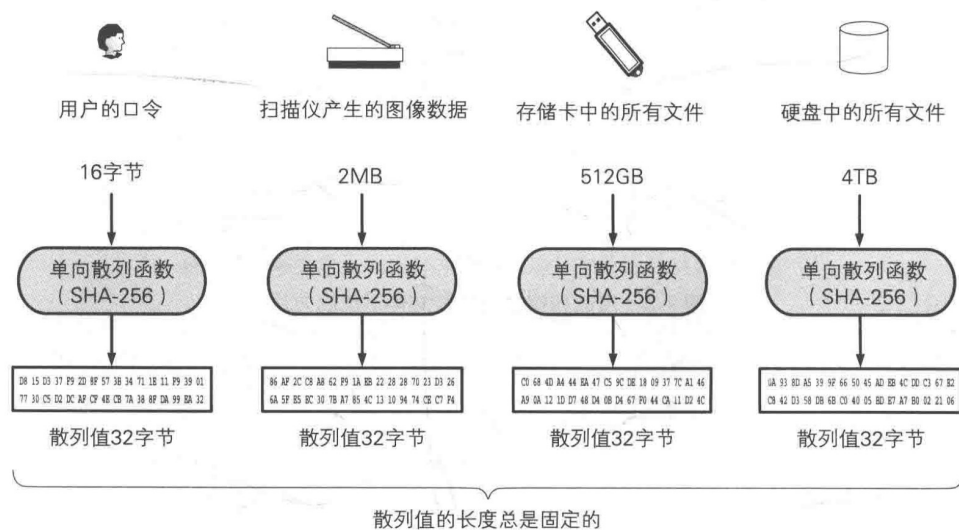


4.1 单向散列函数

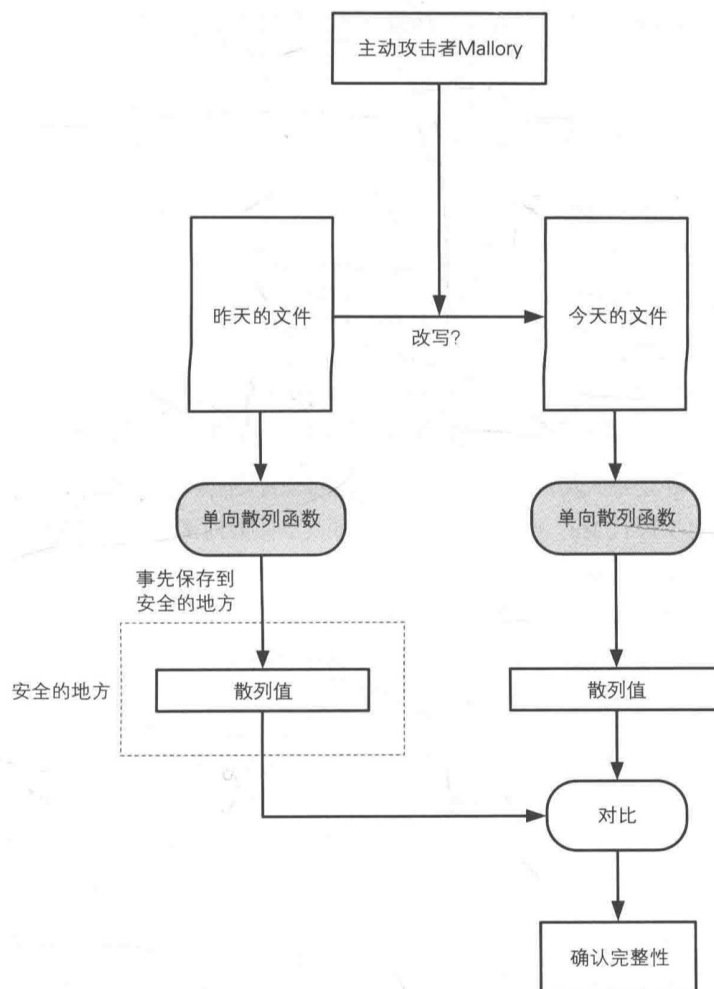
- 单向散列函数 (one-way hash function) 有一个输入和一个输出，其中输入称为消息 (message)，输出称为散列值 (hash value) 。
- 消息不一定是人类能够读懂的文字，也可以是图像文件或者声音文件。单向散列函数不需要知道消息实际代表的含义。
- 散列值的长度和消息的长度无关。单向散列函数都会计算出固定长度的散列值。SHA-256 单向散列所计算出的散列值的长度永远是256 比特（32字节）。

4.1 单向散列函数的性质

- 根据任意长度的消息计算出固定长度的散列值
- 能够快速计算出散列值
- 消息值不同散列值也不同（抗碰撞性）
- 单向散列函数必须具备单向性



4.1 单向散列函数的应用



4.1 变色龙哈希

- 传统哈希函数是很难找到碰撞的。变色龙哈希函数可以人为设下一个“弱点”：掌握了它就能轻松找到碰撞。
- 思考：为什么需要弱化的哈希函数？



局部敏感哈希

- 局部敏感哈希（Locality-Sensitive Hashing, LSH）是一种用于高维数据近似最近邻搜索的哈希技术。与传统的哈希函数不同，LSH 的目标不是避免冲突，而是最大化相似数据点之间的冲突概率，同时最小化不相似数据点之间的冲突概率。
- LSH 的应用场景
 - 近似最近邻搜索：在图像、音频、视频数据库中查找相似内容。
 - 推荐系统：寻找与用户历史行为相似的其他用户或物品（协同过滤）。
- 优点与缺点
- 优点：速度快：查询时间通常为次线性，适合海量数据。理论保证：许多 LSH 族有严格的碰撞概率界，可证明是近似最近邻的优良方案。可扩展性：易于分布式实现，适用于大规模并行计算。
- 缺点：近似结果：不能保证返回精确最近邻，但可以通过调整参数平衡精度与速度。参数敏感：需要根据数据分布和相似度阈值调整哈希函数个数、桶大小等参数。内存占用：需要存储多个哈希表，可能占用较多内存，但可以通过索引结构优化。

4.1 单向散列函数举例

- MD5: 能够产生128比特的散列值。
 - 2004年已被山东大学王小云教授团队攻破
- SHA256, SHA512: 能够产生256和512比特的散列值。
 - SHA256在区块链中被使用
- 单项散列函数无法解决的问题: **可以辨别篡改, 无法辨别伪装。**
 - 检查完整度是不够的, 还需要进一步的认证

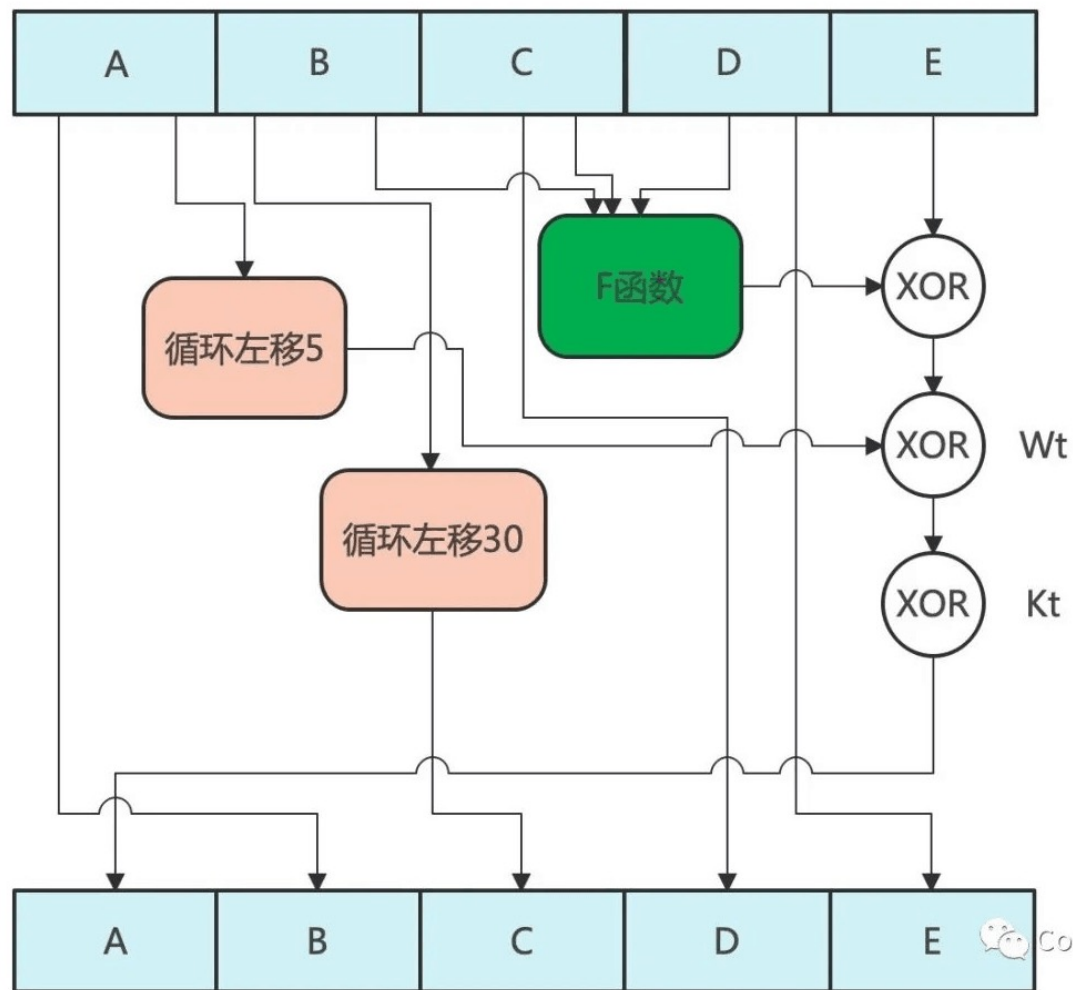
4.1 单向散列函数举例-SHA1

- SHA-1 (Secure Hash Algorithm 1) 是一种密码散列函数, 美国国家安全局设计, 并由美国国家标准技术研究所 (NIST) 发布为联邦资料处理标准 (FIPS)。SHA-1 可以生成一个被称为消息摘要的160位 (20字节) 散列值, 散列值通常的呈现形式为40个十六进制数。
- 消息填充(Message Padding): 数据填充(Append Padding Bits): SHA-1 是按照分块进行处理的, 分块长度为512bit, 大多数情况下, 数据的长度不会恰好满足是512的整数倍, 因此需要进行「padding」到给定的长度。



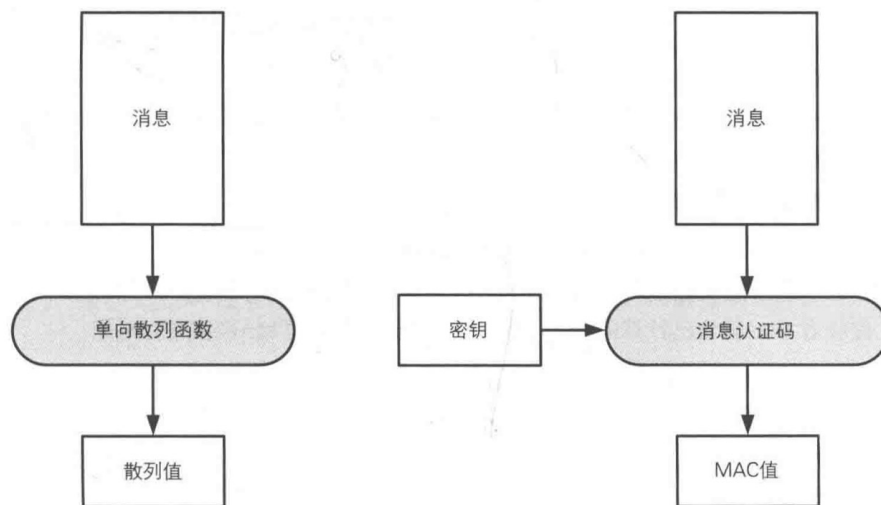
4.1 单向散列函数举例-SHA1

□ 计算摘要



4.2 消息认证码

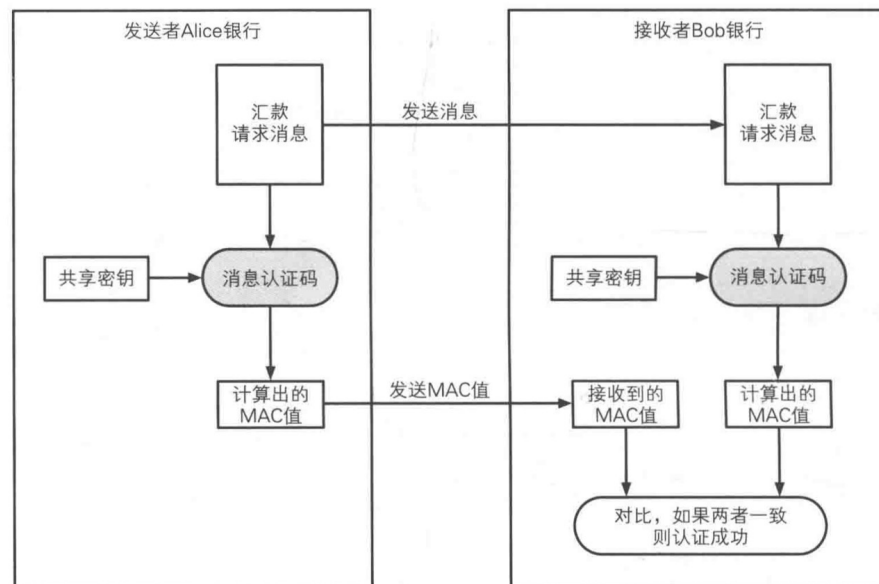
- ❑ 消息认证码(Message Authentication Code) 是一种确认完整性并进行认证的技术，简称为 MAC。
- ❑ 消息认证码的输入包括任意长度的消息和一个发送者与接收者之间共享的密钥，它可以输出固定长度的数据，这个数据称为 MAC 值。
- ❑ 消息认证码是一种与密钥相关联的单向散列函数



4.2 消息认证码

1. 发送者 Alice 与接收者 Bob 事先共享密钥。
2. 发送者 Alice 根据汇款请求消息计算 MAC 值（使用共享密钥）。
3. 发送者 Alice 将汇款请求消息和 MAC 值两者发送给接收者 Bob。
4. 接收者 Bob 根据接收到的汇款请求消息计算 MAC 值（使用共享密钥）。
5. 接收者 Bob 将自己计算的 MAC 值与从 Alice 处收到的 MAC 值进行对比。
6. 如果两个 MAC 值一致，则接收者 Bob 就可以断定汇款请求的确来自 Alice（认证成功）；如果不一致，则可以断定消息不是来自 Alice（认证失败）。

第 13 页



消息认证码可以确认消息是否被篡改，
以及是否有人伪装成消息发送者

注意：如果共享密钥落入攻击者手中，
则攻击者也可以计算出 **MAC** 值，这样
一来消息认证码就无法发挥作用。



4.2 消息认证码的应用实例

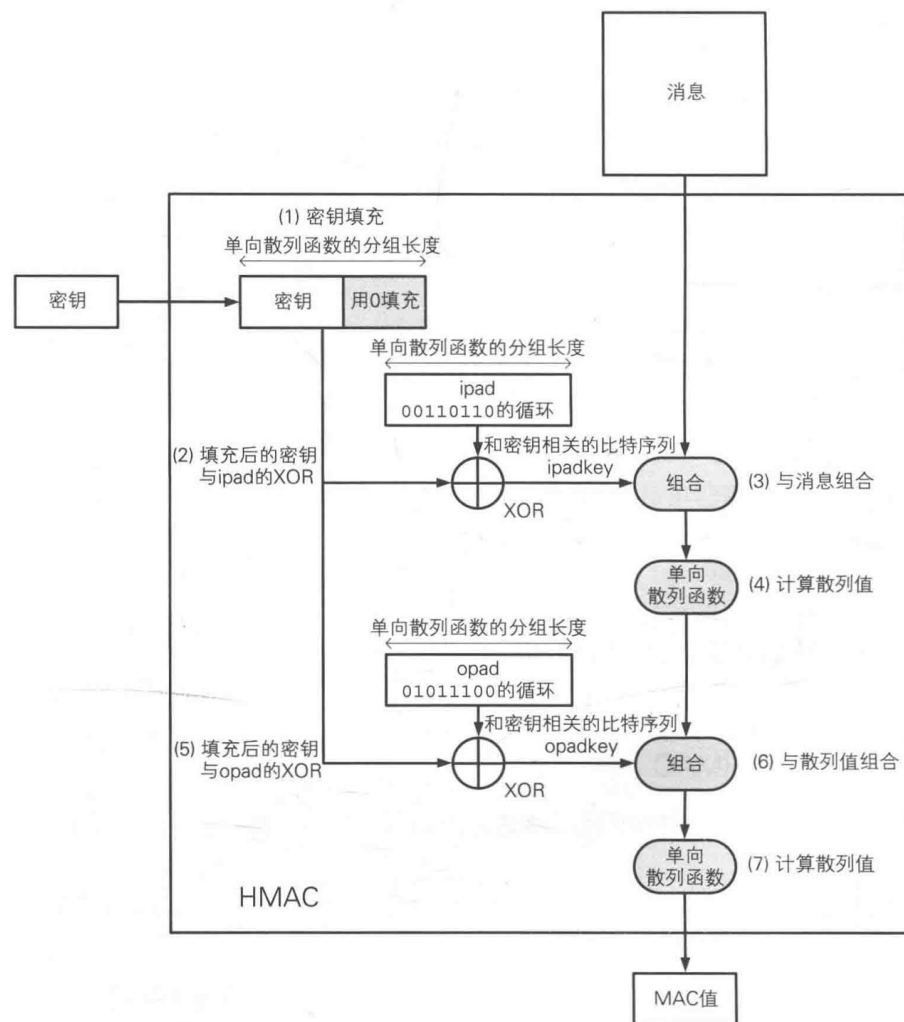
- ❑ SWIFT (Society for Worldwide Interbank Financial Telecommunication, 环球银行金融电信协会), 是于1973 年成立的组织, 其目的是为国际银行间的交易保驾护航。该组织成立时有 15个成员国, 2008 年时已经发展到208 个成员国。银行和银行之间是通过SWIFT 来传递交易消息。而为了确认消息的完整性以及对消息进行验证, SWIFT 中使用了消息认证码。消息认证码所使用的共享密钥都是由人来进行配送的。
- ❑ IPsec 是对互联网基本通信协议IP协议 (Internet Protocol)增加安全性的一种方式。在IPsec 中, 对通信内容的认证和完整性校验都是采用消息认证码来完成的。
- ❑ SSL/TLS 是我们在网上购物等场景中所使用的通信协议。SSL/TLS 中对通信内容的认证和完整性校验也使用了消息认证码。

4.2 消息认证码的实现

- ❑ MAC有多种实现方法，包括使用单向散列函数，公钥密码，流密码等。
- ❑ HMAC: HMAC 是一种使用单向散列函数来构造消息认证码的方法。其中 HMAC的H就是 Hash 的意思。
- ❑ HMAC 中所使用的单向散列函数并不仅限于一种，任何高强度的单向散列函数都可以被用于HMAC，如果将来设计出新的单向散列函数，也同样可以使用。

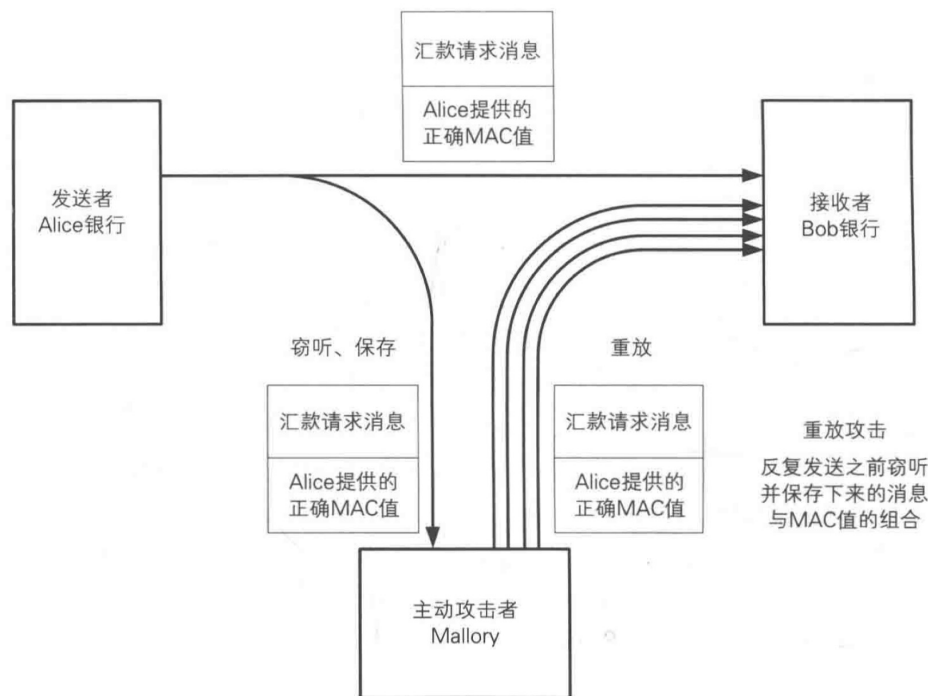
4.2 消息认证码的实现（HMAC）

- ❑ ipad 是将00110110这一比特序列（即16进制的 36）不断循环反复直到达到分组长度所形成的比特序列，其中ipad的i是inner（内部）的意思。
- ❑ opad 是将 01011100 这一比特序列（即16进制的 5C）不断循环反复直到达到分组长度所形成的比特序列，其中opad的。是outer（外部）的意思。



4.2 消息认证码的攻击

- 攻击者并没有破解消息认证码，而只是将正确的MAC 值保存下来重复利用。这种攻击方式称为重放攻击 (replay attack) 。
- 防御手段：序号，时间戳
 - 约定每次都对发送的消息赋予一个递增的编号（序号），并且在计算 MAC 值时将序号也包含在消息中。
 - 约定在发送消息时包含当前的时间（时钟必须一致）



4.2 消息认证码无法解决的问题

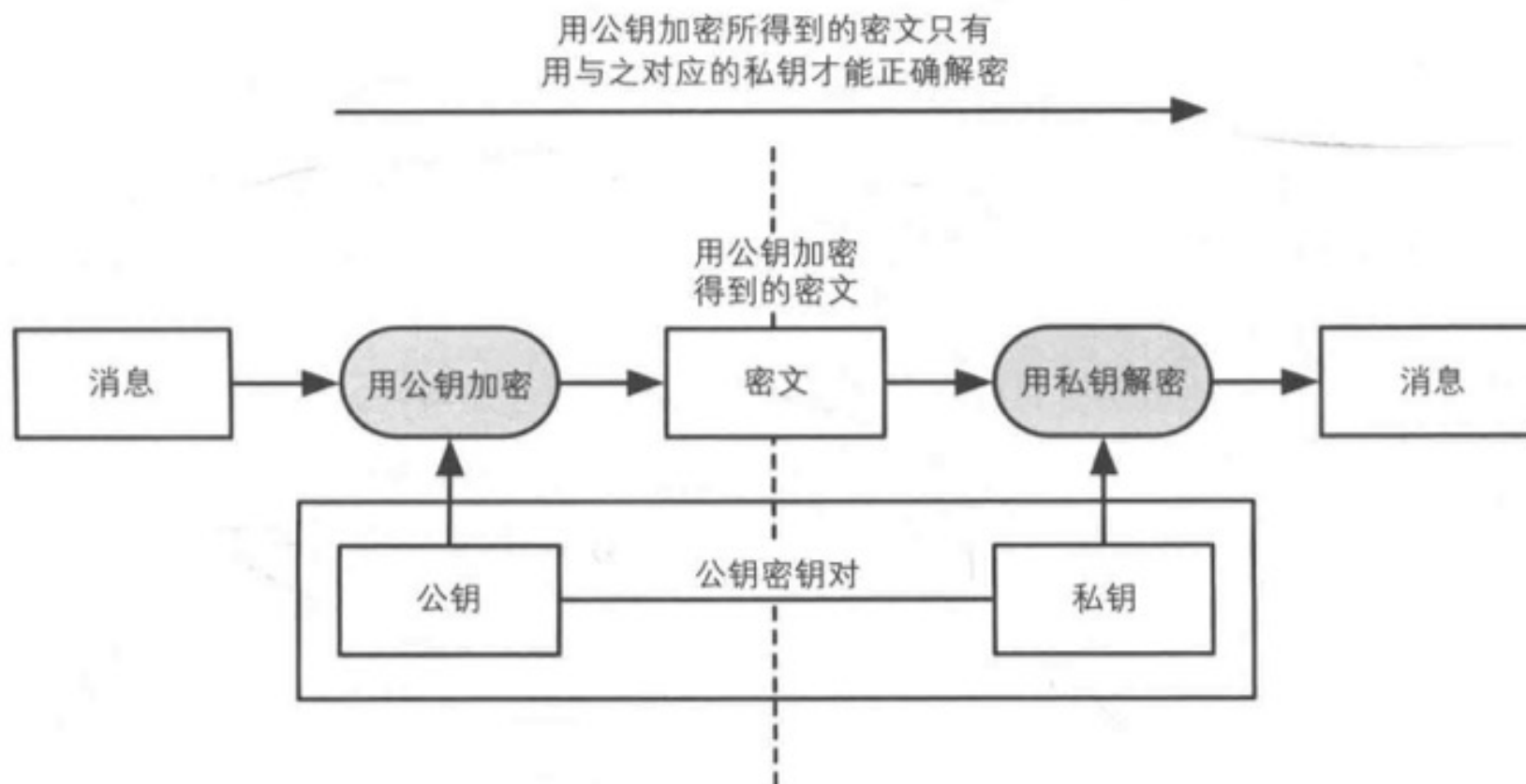
- 收到的消息来自于消息的发送方，并且数据未被篡改。
- **对第三方证明**：假设 Bob 在接收了来自 Alice 的消息之后，想要向第三方验证者 Victor 证明这条消息的确是 Alice 发送的，但是用消息认证码无法进行这样的证明（可能 Bob 发送）。
- **防止否认**：发送者 Alice 可以向 Victor 声称：“我没有向 Bob 发送过这条消息。这样的行为就称为否认 (repudiation)。”
- **根本问题**：消息到底是谁写的？

4.3 数字签名

- 消息认证码的局限性：消息认证码无法防止否认
 - 发送方和接收方共享一个密钥
- 通过数字签名解决问题：发送方和接收方使用不同的密钥
- 我们经常会在邮件的末尾附上一段文字来表明自己的名字和所在的公司等信息，这些文字也称为“签名”。邮件的签名和本章所说的数字签名是完全不同的两码事。在邮件末尾添加的签名是一串固定的文字，而本章所说的数字签名则是根据消息内容生成的一串“只有自己才能计算出来的数值”，因此数字签名的内容是随消息的改变而改变的。

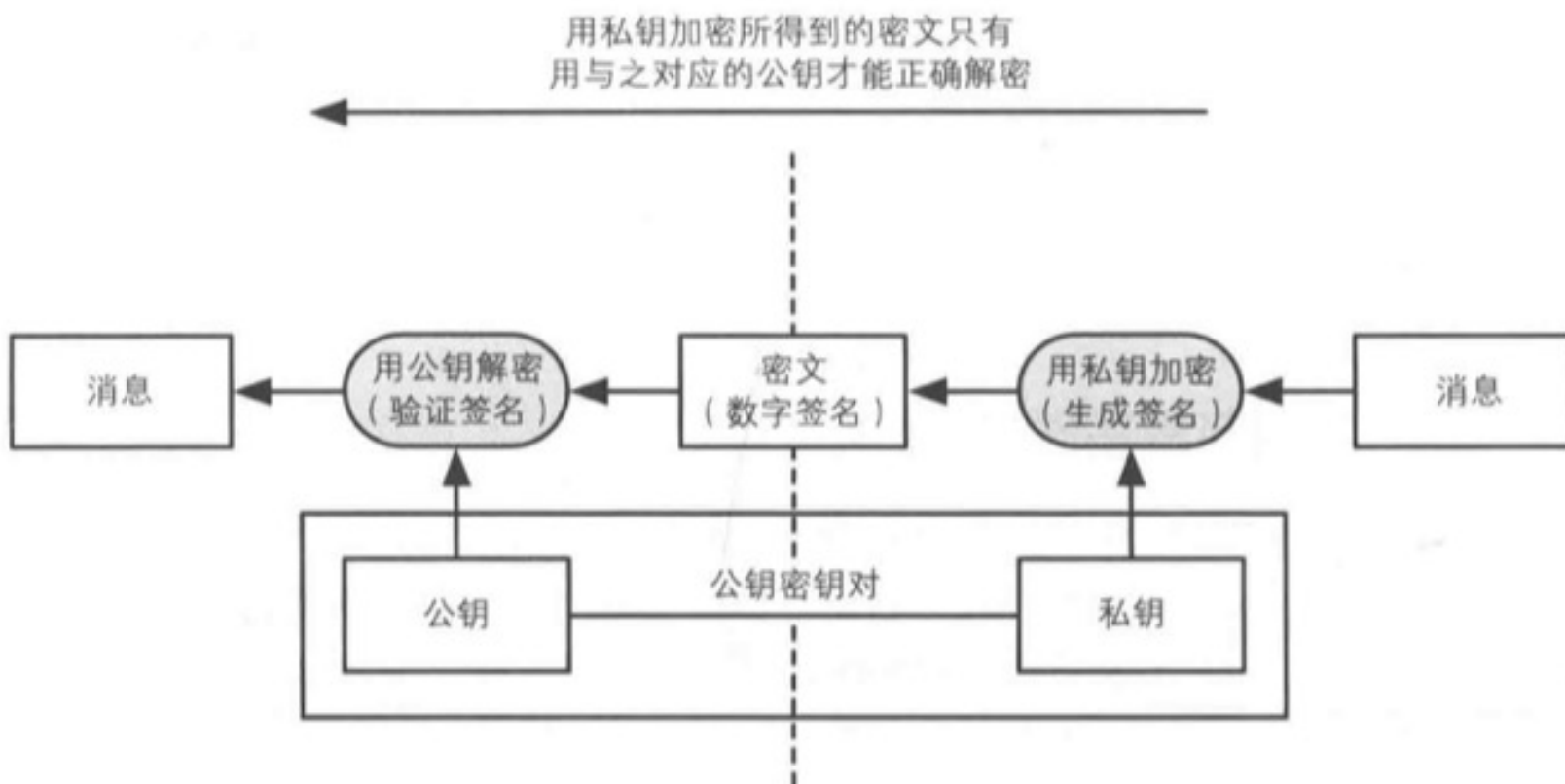
4.3 数字签名

□ 公钥密码



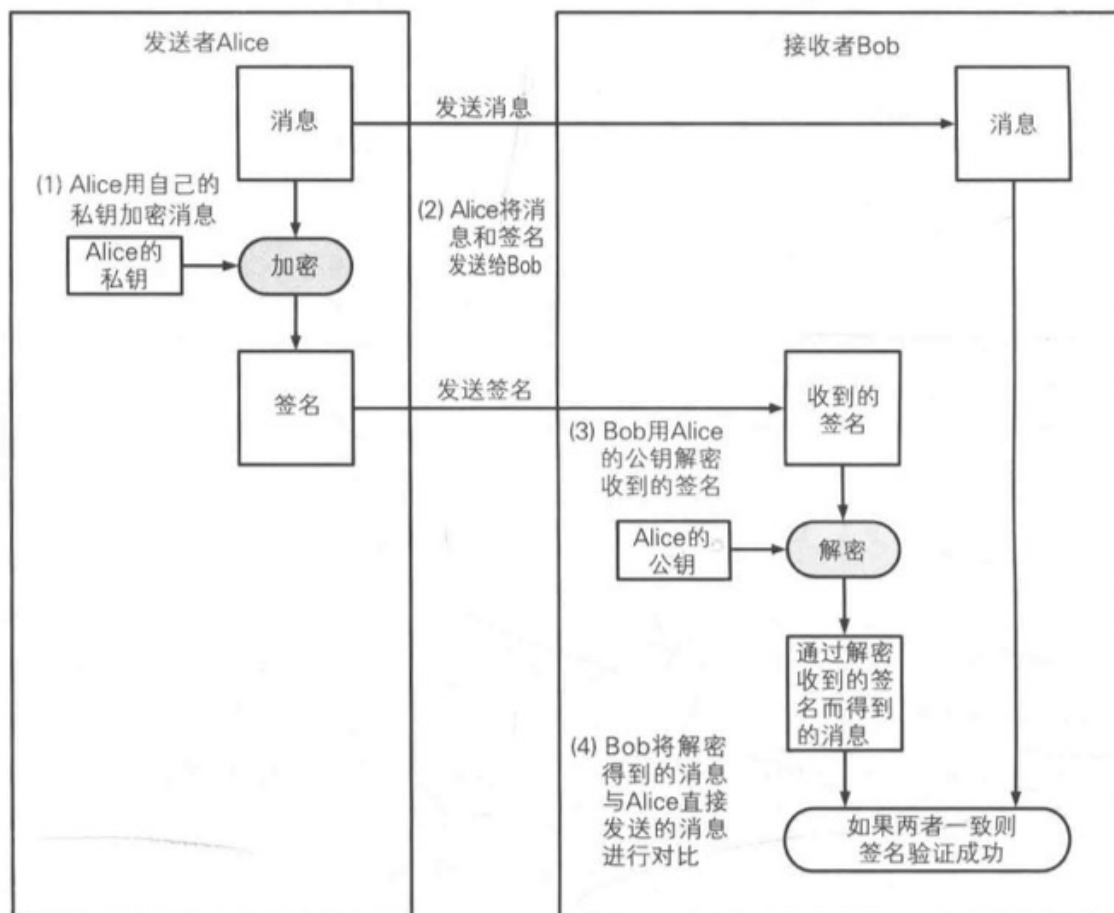
4.3 数字签名

- 数字签名：私钥加密生成签名，公钥解密验证签名（能解密必然是使用了与公钥配对的私钥）



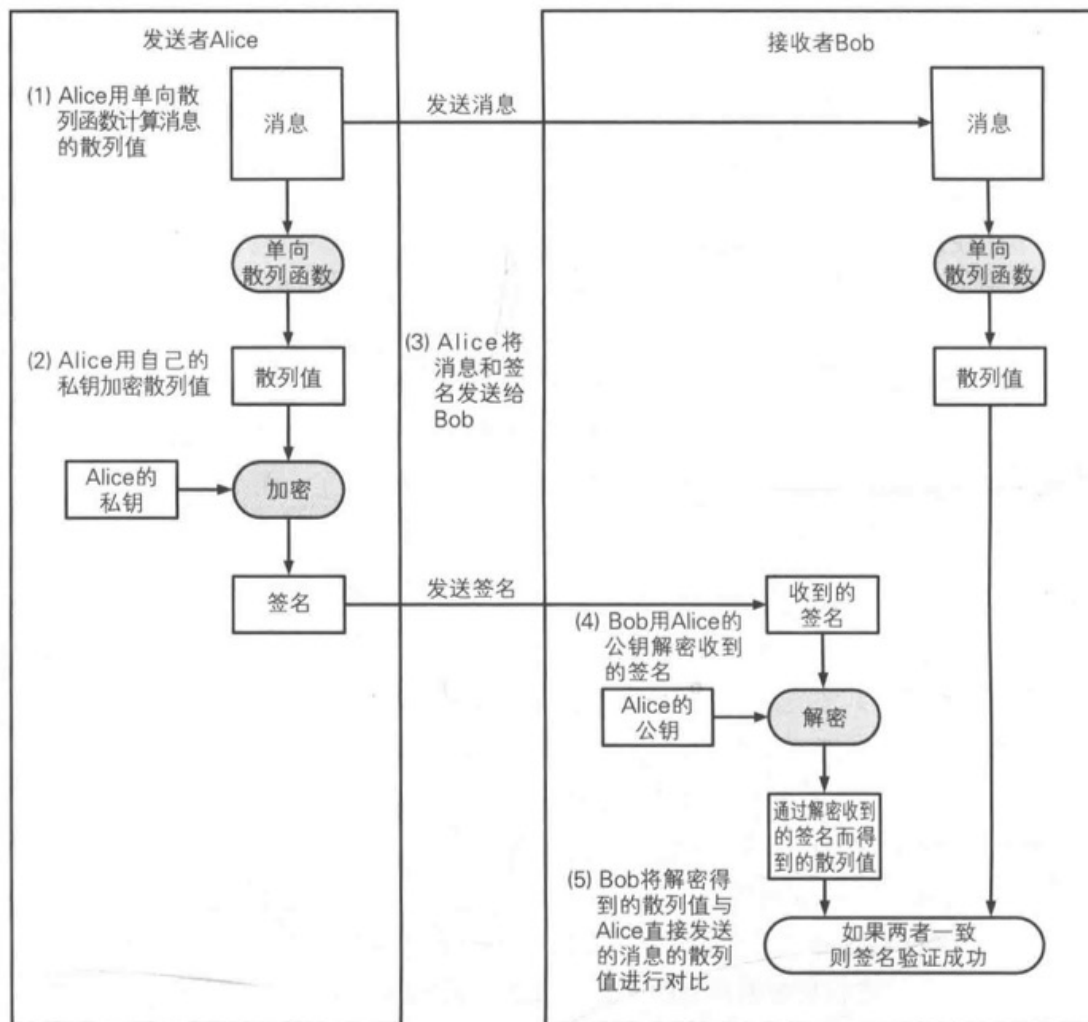
4.3 数字签名的方法

□ 直接对消息签名



4.3 数字签名的方法

□ 对消息散列值签名



4.3 数字签名Q/A

- ❑ 密文为什么能够具备签名的意义呢？
 - 数字签名是利用了“没有私钥的人事实上无法生成使用该私钥所生成的密文”这一性质来实现的。这里所生成的密文并非被用于保证机密性，而是被用于代表一种只有持有该密钥的人才能够生成的信息。
- ❑ 消息没有经过加密就发送了，这样不就无法保证消息的机密性了吗？
 - 数字签名的作用本来就不是保证机密性。
- ❑ 消息认证码无法防止否认，为什么数字签名就能够防止否认呢？
 - 防止否认与“谁持有密钥”这一问题密切相关。

4.3 数字签名的应用

□ 安全信息公告（明文签名）

消息	{	<pre>-----BEGIN PGP SIGNED MESSAGE----- Hash: SHA256 各位 JPCERT-AT-2015-0011 JPCERT/CC 2015-04-15(初版) 2015-04-16(更新) <<< JPCERT/CC Alert 2015-04-15 >>> 请注意Adobe Flash Player安全漏洞 (APSB15-06) Phishing frauds targeting Japanese organizations https://www.jpcert.or.jp/at/2015/at150011.html I. 概要 Adobe Flash Player 中存在一些漏洞。第三方可能会利用这些漏洞，远程诱导用户打开一些经过特殊处理的内容，造成 Adobe Flash Player 异常终止，或者远程执行任意代码。关于漏洞的详情，请参见 Adobe Systems 公司发布的信息。(以下省略) -----BEGIN PGP SIGNATURE-----</pre>
		<pre>iQEcBAEBCAAGBQJVL1dNAAoJEDF916Rp7OBIGQ4H/2qCrsDw5lgX/6/urmYZjfTn 82YgIXnEE0TmOOxDuom+/Sp6E8LF7LHKvR5Mh065mDZ8mqfzGgJGHn+nFTdUYSW0 CHInW9R4rBdRXICpdo6yRQ7BaZai6Oy52glid5eYcf5i8TFmCNLiRIP7tFgqI2ox E57j3mApHY60bYs9KxgNXQ3o3UxzzwDUMU5D49iMXvwccyVbje1IsV0BOKLZX+/7 F8z0i1M5I2P6yofKwBSQCPeMtE+uuInt4IBV9UnJ6dnClluceufUs29bT/OiBpEN 3U9V3HKKSuEso8OKMGjMHUwm1F0PS0675rgj70ezKKAQ8N9MaN0BI5RpuAC1A30= =dZ01 -----END PGP SIGNATURE-----</pre>
数字签名	{	

4.3 密码技术对比

	对称密码	公钥密码
发送者	用共享密钥加密	用公钥加密
接收者	用共享密钥解密	用私钥解密
密钥配送问题	存在	不存在，但公钥需要另外认证
机密性	○	○

	消息认证码	数字签名
发送者	用共享密钥计算 MAC 值	用私钥生成签名
接收者	用共享密钥计算 MAC 值	用公钥验证签名
密钥配送问题	存在	不存在，但公钥需要另外认证
完整性	○	○
认证	○ (仅限通信对象双方)	○ (可适用于任何第三方)
防止否认	×	○

4.3 变色龙哈希

❑ 电子签名的信任问题：信任不能转到第三方（non-transferability）

➤ 假设小明和小红达成协议，小明将家族产业10%股份转让给小红。签了合同小明很担心，如果小红把这件事告诉别人怎么办？这时候就可以用变色龙函数进行签名。小红生成一个只有她才能找到碰撞的函数交给小明，小明再用这个函数来签名。这下小红把签名后的文件丢给大家也没人相信她了。

4.3 变色龙哈希

- ❑ 电子签名的信任问题：信任不能转到第三方（non-transferability）
 - 假设小明和小红达成协议，小明将家族产业10%股份转让给小红。签了合同小明很担心，如果小红把这件事告诉别人怎么办？这时候就可以用变色龙函数进行签名。小红生成一个只有她才能找到碰撞的函数交给小明，小明再用这个函数来签名。这下小红把签名后的文件丢给大家也没人相信她了。
 - 小红掌握着哈希函数的弱点，可以随便生成哈希碰撞。她把10%股份改成99%都能保证哈希不会变，进而创造出新文件本来就是小明签的这种假象。因此出自小红之手的文件可信度为零。

4.3 变色龙哈希

- 如果小明抵赖如何解决？
 - 如果小明死死咬定转让10%股份是小红伪造，事实上只有1%呢？
- 事实上小明也不能信口开河，他得提供对应的证据：哈希碰撞。
 - 假设双方达成的最初合同是A，而小红将其篡改成了相同哈希的A'，那么小明看到A'这份伪证之后一定能拿出最早那个A来并表明A和A'形成哈希碰撞，否则A'就是真货了。因为正常情况下小明无论如何也找不到碰撞，他就不能抵赖。这个特性叫non-repudiation。

4.3 数字签名无法解决的问题

- 用数字签名既可以识别出篡改和伪装，还可以防止否认。
- （思考）数字签名的问题？

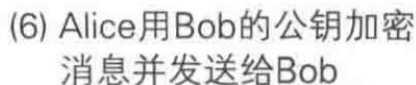
4.3 数字签名无法解决的问题

- 用数字签名既可以识别出篡改和伪装，还可以防止否认。
- 要正确使用数字签名，有一个大前提，那就是用于验证签名的公钥必须属于真正的发送者。即便数字签名算法再强大，如果你得到的公钥是伪造的，那么数字签名也会完全失效。

4.4 证书

- 公钥证书 (Public-Key Certificate, PKC), 由**认证机构** (Certification Authority, CA)施加数字签名。只要看到公钥证书, 我们就可以知道认证机构认定该公钥的确属于此人, 公钥证书也简称为**证书** (certificate)。
- 认证机构必须是可信的, 即可信的第三方。

- (3) 认证机构Trent用自己的私钥对Bob的公钥施加数字签名并生成证书



4.4 证书标准规范

- 证书是由认证机构颁发的，其中使用最广泛的是由 ITU (International Telecommunication Union, 国际电信联盟) 和 ISO (International Organization for Standardization, 国际标准化组织) 制定的 X.509 规范 (RFC3280)。很多应用程序都支持 X.509 并将其作为证书生成和交换的标准规范。

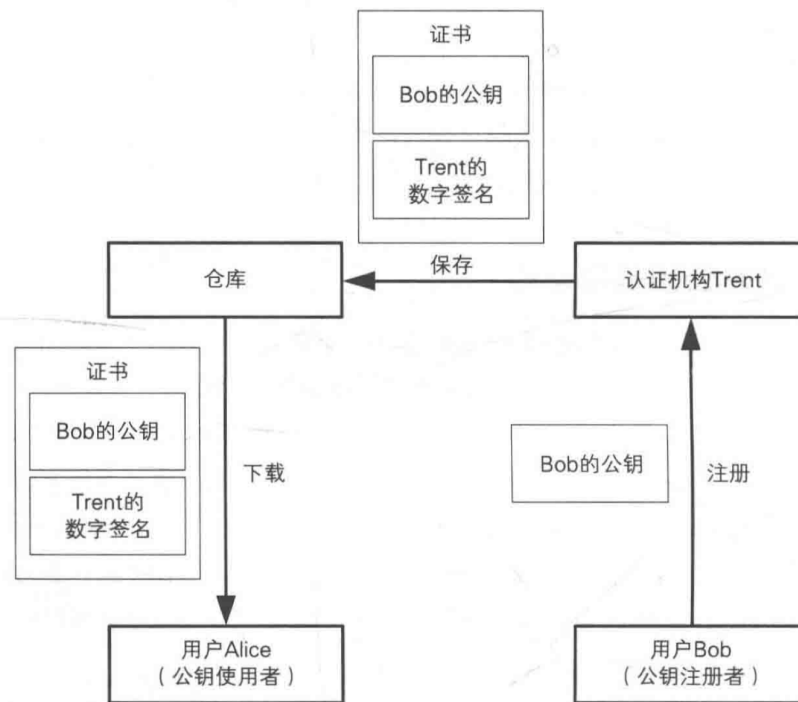
证书序列号	S/N: 24F1FD364C078EA7EDAC7886F0FF6DB2
证书颁发者	Issuer: CN=Symantec Class 1 Individual Subscriber CA - G4,...
公钥所有者	Subject: ... aka: <bobby@example.com>
SHA-1 指纹	sha1_fpr: 39:28:FE:1B:6F:CE:42:F7:D6:E1:95:CF:71:57:C6:0F:C1:2F:A0:E1
MD5 指纹	md5_fpr: 95:2B:83:B5:FF:E8:02:EF:9D:AB:EC:30:37:D2:39:FD
证书 ID	certid: 2BE81DBC305B0007345579C660DC6FEC5DC216EE.24F1FD364C078EA7EDAC7886F0FF6DB2
有效期 (起始时间)	notBefore: 2015-06-12 00:00:00
有效期 (结束时间)	notAfter: 2015-07-07 23:59:59
散列算法	hashAlgo: 1.2.840.113549.1.1.5 (sha1WithRSAEncryption)
密钥类型	keyType: 2048 bit RSA
密钥 ID	subjKeyId: EAD8D26B0695D66A62D91D95C794BD679CB4C740
密钥用途	keyUsage: digitalSignature keyEncipherment

4.4 公钥基础设施（PKI）

- ❑ 仅制定证书的规范还不足以支持公钥的实际运用，我们还需要很多其他的规范，例如证书应该由谁来颁发，如何颁发，私钥泄露时应该如何作废证书，计算机之间的数据交换应采用怎样的格式等。
- ❑ 公钥基础设施(Public-Key Infrastructure, PKI)是为了能够更有效地运用公钥而制定的一系列规范和规格的总称。
 - PKI只是一个总称，而并非指某一个单独的规范或规格。
 - RSA公司所制定的 PKCS (Public-key Cryptography Standards, 公钥密码标准) 系列规范也是PKI 的一种。
 - 互联网规格 RFC (Request for Comments) 中也有很多与 PKI 相关的文档。
 - 之前提到的 X.509 这样的规范也是 PKI 的一种。

4.4 PKI的组成要素

- 用户：使用 PKI 的实体
 - 希望使用 PKI 注册自己的公钥的人
 - 生成密钥对（也可以由认证机构生成）
 - 在认证机构注册公钥
 - 向认证机构申请证书
 - 根据需要申请作废已注册的公钥
 - 解密接收到的密文
 - 对消息进行数字签名
 - 希望使用已注册的公钥的人
 - 将消息加密后发送给接收者
 - 验证数字签名
- 认证机构：颁发证书的实体
- 仓库：保存证书的数据库



4.4 认证机构的工作

□ 生成密钥对

➤ 成密钥对有两种方式：一种是由 PKI 用户自行生成，一种是由认证机构来生成。在认证机构生成用户密钥对的情况下，认证机构需要将私钥发送给用户。

□ 注册证书

➤ 在用户自行生成密钥对的情况下，用户会请求认证机构来生成证书。认证机构根据其认证业务准则 (Certification Practice Statement, CPs) 对用户的身份进行认证，并生成证书。在生成证书时，需要使用认证机构的私钥来进行数字签名。生成的证书格式是由 X.509 定义的。

□ 作废证书与 CRL

➤ 当用户的私钥丢失、被盗时，认证机构需要对证书进行作废。

➤ 用户会保存证书的副本，但认证机构又不能入侵用户的电脑将副本删除。要作废证书，认证机构需要制作一张证书作废清单(Certificate Revocation List, CRL)。

➤ CRL 是认证机构宣布作废的证书一览表，具体来说，是一张已作废的证书序列号的清单，并由认证机构加上数字签名。证书序列号是认证机构在颁发证书时所赋予的编号，在证书中都会记载。

➤ PKI 用户需要从认证机构获取最新的 CRL，并查询自己要用于验证签名（或者是用于加密）的公钥证书是否已经作废。

4.4 证书Q/A

□ 为什么需要证书？

➤ 在通过不可信的途径（例如邮件）获取公钥时，可能会遭到中间人攻击。如果从认证机构获取公钥，就可以降低遭到中间人攻击的风险。因为带有证书的公钥是经过认证机构进行数字签名的，事实上无法被篡改。

□ 使用公司自己开发的保密的认证方法更安全吧？

➤ 错误的。自己开发保密的认证方法是犯了典型的隐蔽式安全（security by obscurity）错误。私下开发安全相关的技术是非常危险的。仅靠一家公司的力量无法开发出足以抵御攻击的安全技术，这一点不仅限于密码技术，对于数字签名和证书等认证技术也同样适用。

4.5 密码技术的完美组合：PGP

- ❑ PGP 是于 1990 年左右由菲利普·季默曼（Philip Zimmermann）个人编写的密码软件，现在依然在世界上被广泛使用。PGP 这个名字是 Pretty Good Privacy(很好的隐私)的缩写。
- ❑ PGP 可以在 Windows、Mac OS X、Linux 等很多平台上运行，版本包括商用版和免费版。
- ❑ 此外还有一个由 GNU 遵照 OpenPGP（RFC4880）规范编写的叫作 GnuPG（GNU Privacy Guard）的自由软件。

4.5 PGP的功能

- ❑ 对称密码：支持用对称密码进行加密与解密。算法包括：AES, 3DES, IDEA等。
- ❑ 公钥密码：支持生成公钥密码的密码对，以及用公钥密码进行加密与解密。支持的算法包括RSA, ElGamal等。
- ❑ 数字签名：支持数字签名的生成和验证。算法包括：RSA, DSA等。
- ❑ 单向散列函数：算法包括SHA-1, SHA-256, SHA-512等。MD5可以使用，但不推荐。
- ❑ 证书：可生成OpenPGP中规定格式的证书，以及与X.509规范兼容的证书。此外还可以颁发公钥的作废证明，并可以使用CRL对证书进行校验。
- ❑ 压缩：支持对数据的压缩，压缩采用ZIP等格式。
- ❑ 文本数据：支持二进制数据与文本数据互相转换。
- ❑ 大文件的拆分与合并。
- ❑ 钥匙串管理：管理密钥与外部获得的公钥。



4.5 生成密钥对

- 要在PGP中进行加密与数字签名，需要先生成密钥对。

```
$ gpg2 --full-gen-key          ←生成密钥对的命令
gpg (GnuPG) 2.1.4; Copyright (C) 2015 Free Software Foundation, Inc.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

Please select what kind of key you want:
  (1) RSA and RSA (default)
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
Your selection? 1          ←选择密钥类型 (这里我们选择加密算法为 RSA, 数字签名算法也为 RSA)
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (2048) 2048          ←设置密钥的比特数
Requested keysize is 2048 bits
Please specify how long the key should be valid.
  0 = key does not expire
  <n> = key expires in n days
  <n>w = key expires in n weeks
  <n>m = key expires in n months
  <n>y = key expires in n years
Key is valid for? (0) 1y          ←设置密钥有效期 (一年)
Key expires at Sat Jun  4 11:13:57 2016 JST
Is this correct? (y/N) y          ←确认有效期

GnuPG needs to construct a user ID to identify your Key.

Real name: Alice          ←输入姓名
Email address: alice@example.com          ←输入邮箱地址
Comment: Alice Liddell          ←输入备注
You selected this USER-ID:
"Alice (Alice Liddell) <alice@example.com>"

Change (N)ame, (C)omment, (E)mail or (O)kay/(Q)uit? O          ←选择 OK
(这里会显示图 13-2 中的口令输入画面)

We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
We need to generate a lot of random bytes. It is a good idea to perform
some other action (type on the keyboard, move the mouse, utilize the
disks) during the prime generation; this gives the random number
generator a better chance to gain enough entropy.
gpg: /Users/alice/.gnupg/trustdb.gpg: trustdb created          ←创建“信任网”的数据库
gpg: key A57FF192 marked as ultimately trusted          ←自己生成的密钥将被设置为“绝对信任”
gpg: directory '/Users/alice/.gnupg/openpgp-revocs.d' created          ←创建存放作废证书的位置
public and secret key created and signed.

gpg: checking the trustdb
gpg: 3 marginal(s) needed, 1 complete(s) needed, PGP trust model
gpg: depth: 0 valid: 1 signed: 0 trust: 0-, 0g, 0n, 0m, 0f, 1u
gpg: next trustdb check due at 2016-06-04
pub  rsa2048/A57FF192 2015-06-05 [expires: 2016-06-04]          ←A57FF192 密钥生成完毕
Key fingerprint = D00A 09E6 0EE8 3680 0E04 2BEE 81AD 4C36 A57F F192
uid          [ultimate] Alice (Alice Liddell) <alice@example.com>
sub  rsa2048/32D4291D 2015-06-05 [expires: 2016-06-04]
```



4.5 生成密钥对

- 生成的公钥可以用文本格式显示出来

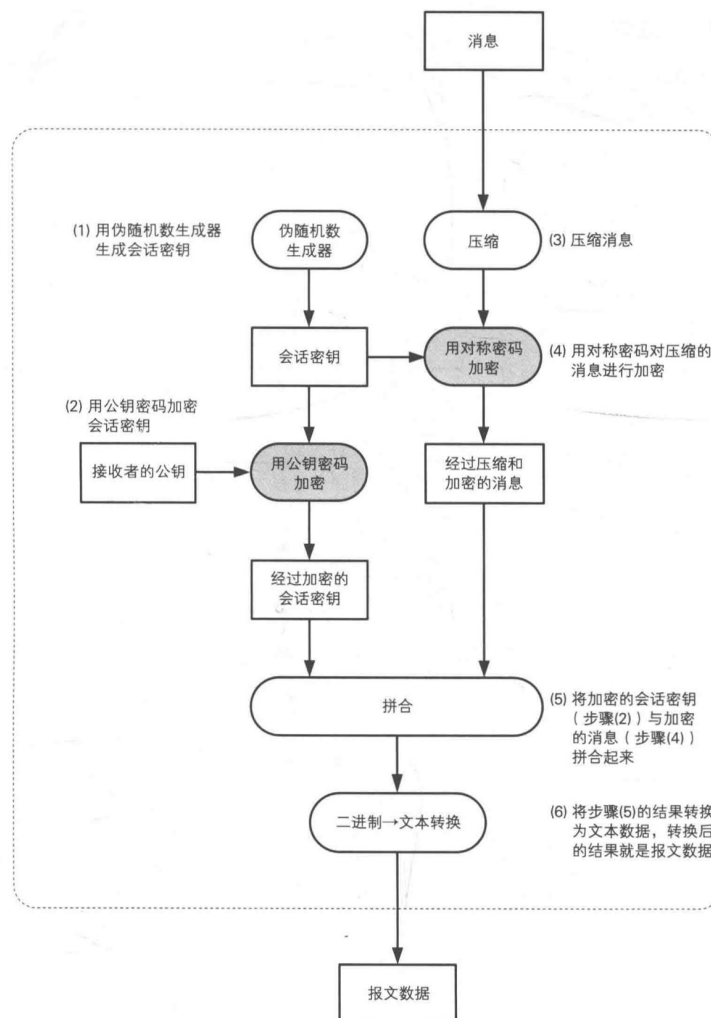
```
$ gpg2 --export --armor A57FF192 ←显示 A57FF192 公钥的内容
-----BEGIN PGP PUBLIC KEY BLOCK-----
Version: GnuPG v2

mQENBFVxBfABCADbmYB9VEg8+/PHXIqFIJ47m4dfEHcnMpr5pYLWUgE/Jp1/UaPJ
jInKntPYAlLD4VvwrgXM8wyxQBua4dvqfFmgFxm8CED4VBCBHuzU/V+YQc/Bjyax
V2zrsWNYyvasZRYucX0ssUb3kQvLITMabi7N1wXITsEUPZsnuLyImZd+hAwd0mEb
rWd+xpeeHa67mHUCDC9HMs7X1rCb2ktAFnvfToEScsbU/DcI3uh54ilzbGnpzHx2
6+isTBkOnsU2bSrCG3r4oR1ShzJjftvhs12RzI7UWqSpb7poID9b126PymBjhtYs
PlyckPHfysZ3b7S0SCpyvPoflX2QCwjMOWzVABEBAAg0KUFsaWN1IChBbGljZSBM
aWRkZWxsKSA8YWxpY2VAZXhhbXBsZS5jb20+iQE9BBMBCAAnBQJvcQXwAhSDBQkB
4TOABQsJCAcCBhUICQoLAgQWAgMBAh4BAheAAoJEIGtTDalf/GSkSsH/Orahwa0
1H1Kt8rdxCrFoZDLQ87tSi+oacL/9UP2ZA/dRRG68tPo6C8pTR8CdIFJKv2s9hXI
yGUEVbIjsiyk1zgiFCihhZu+qNBA3iWNWU63H26TpML9Flik/wYjBXC3ky46shH
CZh6L2mhcfJP3waeHzpxV6Bxg9FTSLVfYqDFly+sSgD1q4TbrCfDQ+7EU4zd0MoP
lNJAG9uqqLTqc4p6q9zdOu6fjroLOPyCU51FVWYqoyRk/mULBJ/Yaf/0s//ZtnX2
IN3UUCijZQvUztbNHWxee21h/u7R+9tGgmTo66qULLkHnzVkiGhM9Y7plrFtHzNS
34c+SNc9tLMsfS+5AQ0EVXEF8AEIAO2hzbilSOIxlLIQaqshuGzPWB9qXGUHIAyT
4E32xMWT8oeWDyUP0fXQUlyZZbndWbJmaYdrHHTfFG5i4vrXClrvSt13taICImDV
H05DnHBQBQyMEwBBJvEenZuBraWgXeg8c/AwQalwY8ahfTizMJsLYRyQToJTe1BN
iv3TrMALHe34wQtn3hq0aC/9WHLvP61PPQU2MIrG+FG94E21DxgksQOX5fCeVFTC
mJQTtiGBQoWzKPr/GRZXBODerBqBnnKWbJkKOnun+RrNrh+sH9ej2VQh2b4YRLQW
Py5AYvCo8rsrp1s3LqkTz3Z1TNBD0QrdqNFODSuLTshYfdyJh8sAEQEAAyKBjQQY
AQgADwUCVXEF8AIbDAUJAeEzgAAKCRCBruW2pX/xkivyB/91wDnHtvU81v/Q3YY1
PJMkKFD05TubmJ0qJVSskLIjAIocnbqiBoxu02bJzt76xuLJEak3tGT7sskto/zN
bYrbsfETuUKMBH3fr+uHtTod9ebjE2i7cF3bNtOTd0I9D7RyeaMsmjeeXfYJhEFD
6cIYh/7mrE2nQtn2+CtWmluzS+lG1iu5uyhNs3ysvS/zyxjuf9gxMrrZTYMyOBBD
2RCsMBvSiS5gGUUVS8iit8hPG3j0tXDqVxC/CT82nr4sdPaFCJO18jt8g3RBAHRf
NveEajGewzr0sY+GtDfKfUruszLTcLinyPvsnzF6LvRa76bFs2qLPQAoGbXGHBt
5mjM
=0rVX
-----END PGP PUBLIC KEY BLOCK-----
```

4.5 加密与解密-加密

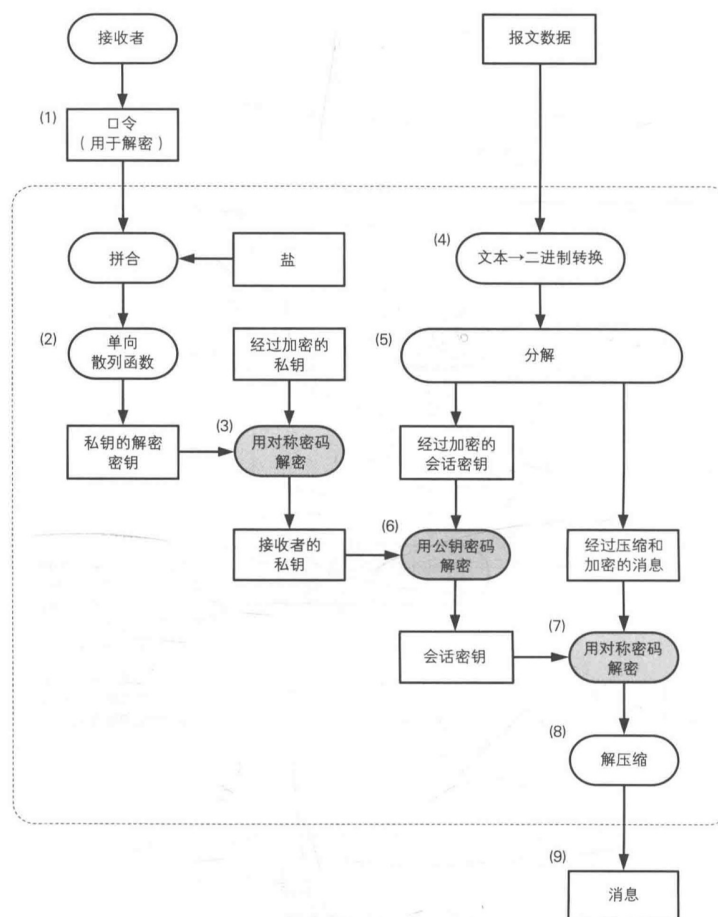
- 生成和加密会话密钥
- 压缩和加密消息
- 用公钥密码加密会话密钥，用对称密码加密消息

思考：消息是先压缩，再加密的，为什么要按照这样的顺序处理？



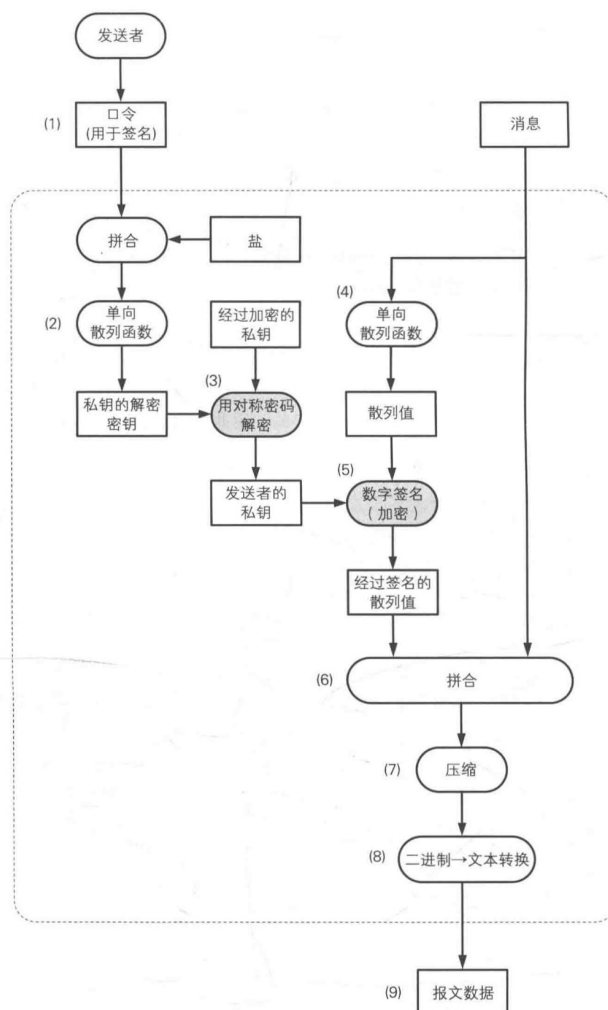
4.5 加密与解密-解密

- 解密私钥
- 解密会话密钥
- 解密和压缩消息



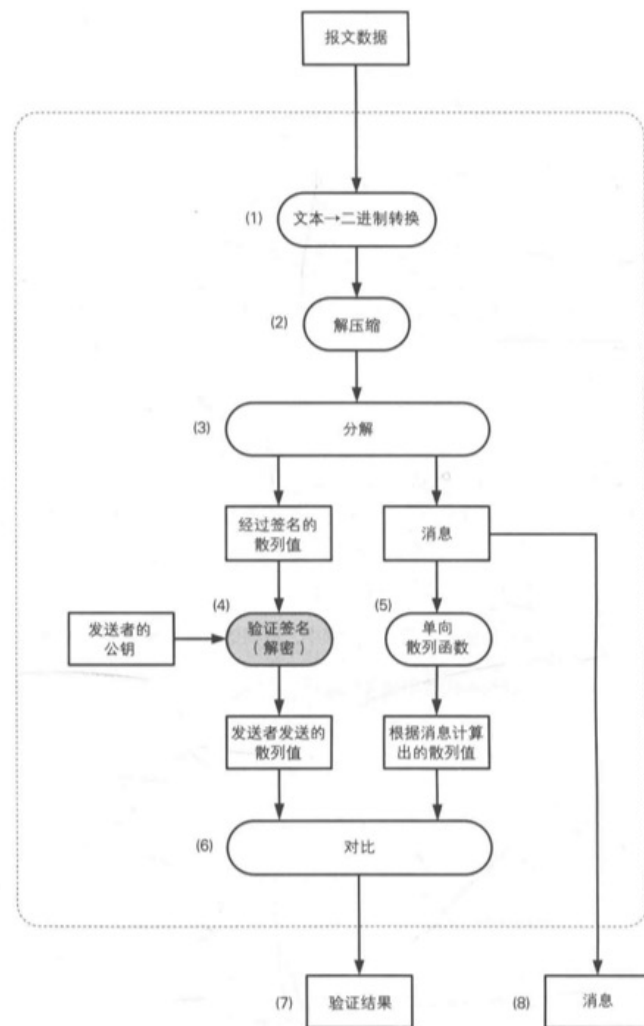
4.5 生成和验证数字签名-生成

- 解密私钥
- 生成数字签名



4.5 生成和验证数字签名-验证

- 恢复发送者发送的散列值
- 对比散列值



4.5 信任网

- 在使用PGP时，确认自己所得到的公钥是否真的属于正确的人（公钥合法性）是非常重要的，因为公钥可能会通过中间人攻击被替换。
- 证书是确认公钥合法性的方法之一。证书就是由认证机构对公钥所施加的数字签名，通过验证这个数字签名就可以确认公钥合法性。
- PGP 中却没有使用认证机构，而是采用了一种叫作信任网(web of trust)的方法。在这种方法中，PGP用户会互相对对方的公钥进行数字签名。
- 信任网的要点是“不依赖认证机构，而是建立每个人之间的信任关系”。换言之，就是能够自己决定要信任哪些公钥。

4.5 信任网的建立

- 通过自己进行确认
- 通过自己完全信任的人进行确认
- 通过自己有限信任的多个人进行确认